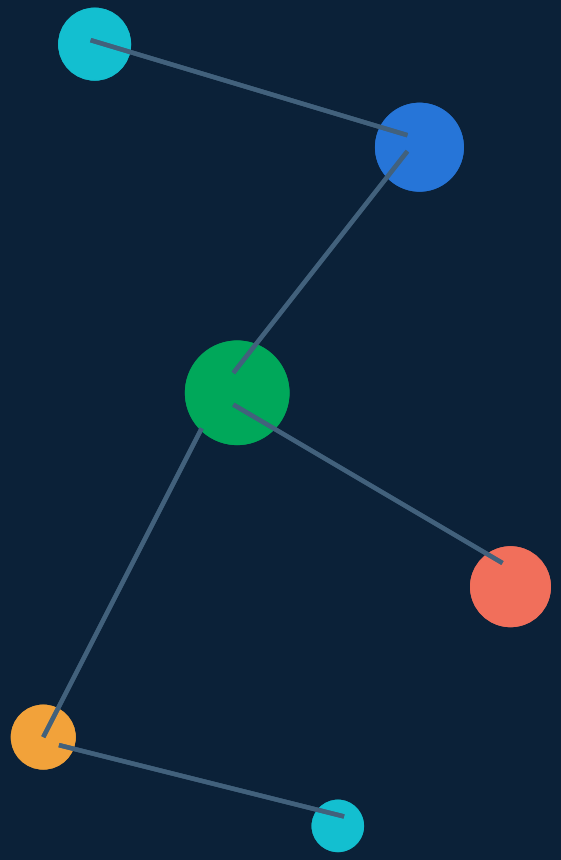
PRÁCTICA 2

Algoritmos codiciosos · Programación dinámica

Cambio de monedas · Selección de actividades · Planificación de tareas

Curso 2026-27

Guía visual para implementar, experimentar y justificar



La práctica en una mirada

Tres problemas, una pregunta común: ¿cuándo basta una decisión local?

1. Cambio de monedas

- greedy_change
- optimal_change
- compare_change

Contrastar rapidez y calidad de la solución.

2. Selección de actividades

- activity_selection
- generate_activities

Maximizar cuántas actividades compatibles caben.

3. Planificación de tareas

- job_scheduling
- generate_jobs

Maximizar el beneficio con plazos e intervalos unitarios.

Implementar · medir · argumentar

Reglas que afectan a la corrección automática

La interfaz también forma parte de la solución.

Firmas exactas

Respeta nombres, parámetros, tipos de retorno y formatos pedidos.

Sin salida por pantalla

Elimina los print de depuración antes de entregar.

Sin pruebas ejecutables

p2.py debe poder importarse sin ejecutar pruebas ni instrucciones adicionales.

Contenido de p2.py

Incluye solo las funciones solicitadas y las importaciones necesarias.

Entrega: Grupo 126 · 9 nov 2026, 23:59 | Grupo 127 · 10 nov 2026, antes de clase

I · Problema del cambio de monedas

Obtener exactamente amount con monedas disponibles en cantidad ilimitada.

Supuestos del enunciado

- Monedas: enteros positivos y sin valores repetidos.
- Puede no existir moneda de valor 1.
- La lista coins puede venir desordenada.

Dos objetivos distintos

Codicioso: elegir la moneda mayor que quepa en lo que falta.

Óptimo: usar el menor número posible de monedas.

```
amount = 7     coins = [5, 3]     → no siempre hay solución exacta
```

La práctica compara cuándo la regla local funciona y cuándo falla frente a programación dinámica.

I.A.1 · greedy_change

La estrategia codiciosa toma repetidamente la moneda de mayor valor que no supere el resto.

```
greedy_change(coins: List[int], amount: int) ->
Tuple[List[int], int]
```

Qué devuelve

(selected, remainder)

selected: monedas elegidas.
remainder: cantidad que queda sin completar.

Solución exacta si remainder == 0.

Pasos mentales

1. Ordena o busca monedas de mayor a menor.
2. Toma una moneda válida y réstala del resto.
3. Repite mientras alguna moneda quepa.
4. Devuelve las monedas y lo que sobra.

```
[50, 20, 10, 5, 2, 1], amount=87  →  ([50, 20, 10, 5, 2], 0)
```

Un cambio exacto puede ser subóptimo

Ejemplo del enunciado: coins = [4, 3, 1], amount = 6.

Resta 6	Resta 2	Resta 1	Total	Alternativa
elige 4 queda 2	elige 1 queda 1	elige 1 queda 0	4 + 1 + 1 3 monedas	3 + 3 2 monedas

Codicioso: ([4, 1, 1], 0) · Óptimo: [3, 3] · Coincide la cantidad, no el número de monedas

I.B.1 · optimal_change

Programación dinámica guarda la mejor solución para cada cantidad más pequeña.

```
optimal_change(coins: List[int], amount: int) -> List[int]
| None
```

Contrato

Devuelve una combinación con el mínimo número de monedas.

Devuelve None si no se puede formar amount.

Para amount == 0, devuelve [].

Matriz de estados

Columnas: cantidades desde 0 hasta amount. Filas: monedas que se permiten usar hasta ese punto.

Como hay monedas ilimitadas, al usar c se consulta la misma fila en a − c.

$D[i,a] = \min(D[i-1,a],\ 1 + D[i,a-c_i])$ · $D[0,0]=0;\ D[0,a>0]=\infty$

Cómo construye el óptimo para amount = 6

Con coins = [4, 3, 1], la tabla se calcula desde 0 hasta 6.

cantidad 0	cantidad 1	cantidad 2	cantidad 3	cantidad 4	cantidad 5	cantidad 6
0 monedas	**1 moneda**	**2 monedas**	**1 moneda**	**1 moneda**	**2 monedas**	**2 monedas**
[]	[1]	[1,1]	[3]	[4]	[4,1]	[3,3]

Reconstrucción

La mejor forma de 6 usa una moneda 3 y el óptimo guardado para 3: [3, 3].

Sin solución exacta

Con coins = [5, 3] y amount = 7, devuelve None. Ninguna secuencia suma 7.

No se permite búsqueda exhaustiva de todas las combinaciones

Ejemplo DP 1 · optimal_change([4, 3, 1], 6)

Cada celda guarda el menor número de monedas usando las denominaciones disponibles en esa fila.

```
D[i,0] = 0; D[0,a] = ∞ para a>0; (∞ significa "cantidad inalcanzable")
D[i,a] = D[i-1,a] para a<cᵢ; D[i,a] = min(D[i-1,a], 1 + D[i,a-cᵢ]) para a>=cᵢ
```

Fils: i (0..3) Moneda / Cols: a (0..6) Cantidad	0	1	2	3	4	5	6
Ninguna (c0 = 0)	0	∞	∞	∞	∞	∞	∞
Hasta 1 (c1 = 1)	0	1	2	3	4	5	6
Hasta 1, 3 (c2 = 3)	0	1	2	**1**	2	3	2
Hasta 1, 3, 4 (c3 = 4)	0	1	2	1	1	2	**2**

Reconstrucción del óptimo

D[3,6] = 2. La moneda 4 daría 1 + D[3,2] = 3; se elige 3. En D[2,3] se añade otro 3. Resultado: [3, 3].

compare_change

Greedy usa [4,1,1] (3 monedas). DP usa 2 [3,3]: cantidad 6 ∈ suboptimal.

La fila final resume todos los tipos de moneda; el valor de la columna 6 es el óptimo.

Ejemplo DP 2 · óptimo posible e imposible

Con coins = [5, 3], la matriz permite resolver amount = 8 y detectar que 7 no se puede formar.

`Filas: ninguna moneda, luego {3}, luego {3,5}  ·  Columnas: cantidades de 0 a 8`

Monedas permitidas	0	1	2	3	4	5	6	7	8
Ninguna	0	∞	∞	∞	∞	∞	∞	∞	∞
Hasta 3	0	∞	∞	1	∞	∞	2	∞	∞
Hasta 3, 5	0	∞	∞	**1**	∞	**1**	2	**∞**	**2**

Cantidad 8: reconstrucción

D[2,8] = 2. Se toma una moneda 5; queda 3, cuyo óptimo es una moneda 3. Resultado: [5, 3].

Cantidad 7: no hay solución

D[2,7] = ∞. Greedy deja resto 2 y ninguna otra combinación suma 7. compare_change clasifica 7 como impossible.

En 8, greedy coincide con el óptimo de 2 monedas: compare_change clasifica 8 como optimal.

I.C.1 · compare_change

Clasifica cada cantidad según greedy y según la existencia de una solución óptima.

```
compare_change(coins: List[int], max_amount: int) ->
Dict[str, List[int]]
```

optimal

Codicioso exacto y usa el mínimo número de monedas.

suboptimal

Codicioso exacto, pero usa más monedas que el óptimo.

greedy_failed

Codicioso no alcanza la cantidad, aunque sí existe combinación exacta.

impossible

No existe combinación exacta con esas monedas.

```
Evalúa todas las cantidades 1, 2, …, max_amount (incluido).
```

I.D · análisis experimental del cambio

Repite compare_change para cada sistema monetario y para cantidades de 1 a 500.

Sistemas que se deben estudiar

[200, 100, 50, 20, 10, 5, 2, 1]
[10, 5, 2, 1]
[25, 10, 4, 1]
[4, 3, 1]

Tabla de resultados

Para cada sistema, indica el número y el porcentaje de cantidades en cada clase: optimal, suboptimal, greedy_failed e impossible.

Comparación temporal

Mide greedy_change y optimal_change. Compara los tiempos para cantidades crecientes.

Discusión

Relaciona eficiencia y calidad: ¿cuánto trabajo ahorra la regla codiciosa y qué solución deja de garantizar?

Incluir tabla resumen y breve análisis en memoria.pdf

Preguntas para la memoria · cambio de monedas

Responde con argumentos apoyados en la estrategia y las medidas.

- ¿Cuál es el criterio codicioso utilizado?
- ¿Funciona correctamente para todos los sistemas monetarios?
- ¿Qué significa que un sistema monetario sea canónico?
- ¿Cuál es la complejidad temporal de la solución codiciosa?
- ¿Cuál es la complejidad temporal y espacial de la programación dinámica?
- ¿Qué ventajas presenta la solución codiciosa aunque no siempre garantice el óptimo?

II · Selección de actividades

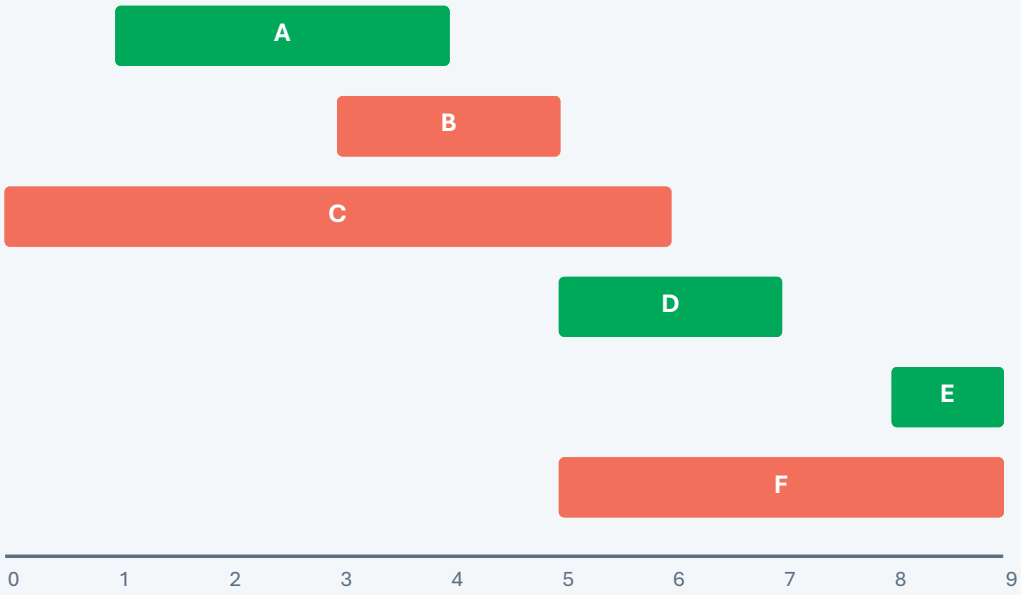
Elegir el mayor número posible de actividades compatibles.

```
Actividad = (identifier, start, end)
```

Compatibilidad

Dos actividades no se solapan. Si una empieza exactamente cuando otra termina, sí son compatibles.

Ejemplo de intervalos



Verde: selección compatible A, D, E. Coral: alternativas que se solapan.

II.A.1 · activity_selection

Ordenar por finalización ascendente; resolver empates por inicio y luego identificador.

```
activity_selection(activities: List[Tuple[str, int, int]]) ->
List[Tuple[str, int, int]]
```

Debe cumplir

- Devuelve una lista nueva.
- No modifica activities.
- Devuelve el máximo número de actividades compatibles.
- La salida queda ordenada según su ejecución.

Selección voraz

1. Ordena por (end, start, identifier).
2. Acepta la primera actividad.
3. Acepta la siguiente si start ≥ end de la última aceptada.

```
[A(1,4), B(3,5), C(0,6), D(5,7), E(8,9), F(5,9)]
→ [A(1,4), D(5,7), E(8,9)]
```

Por qué funciona seleccionar la que termina antes

El argumento de intercambio explica la optimalidad del algoritmo.

1 · Primera elección

Sea g la actividad compatible que termina antes. Sea a la primera actividad de una solución óptima.

2 · Intercambio

Como g termina no después que a, sustituir a por g no bloquea las actividades posteriores de esa solución.

3 · Repetición

Queda el mismo problema sobre las actividades que empiezan después de g. Repetir el razonamiento produce una solución óptima.

Elegir el final más temprano deja el mayor espacio posible para las siguientes actividades.

II.B.1 · generate_activities

Generar datos reproducibles para probar la selección de actividades.

```
generate_activities(n: int, seed: int = 42) -> List[Tuple[str, int, int]]
```

Reglas de generación

- start pertenece a [0, 10000].
- duration pertenece a [1, 100].
- end = start + duration.
- Identificadores: A0, A1, ..., A(n−1).

Para qué sirve seed

La misma semilla permite reproducir la misma entrada.

Separar la generación de la medición ayuda a comparar algoritmos con datos idénticos.

La entrada al selector conserva tuplas (identifier, start, end).

II.C · análisis experimental

Medir el tiempo y la cantidad de actividades seleccionadas.

Tamaños de entrada

100	500	1 000	5 000

Medición

Ejecuta activity_selection para cada tamaño. Repite cada medición temporal al menos 10 veces sobre la misma entrada.

Gráfica 1

Tamaño de entrada frente al tiempo.

Gráfica 2

Tamaño de entrada frente al número seleccionado.

Gráfica 3

Tamaño de entrada frente al porcentaje seleccionado.

Preguntas para la memoria · actividades

Explica la elección, la prueba de optimalidad y los costes.

- ¿Cuál es el criterio codicioso utilizado?
- ¿Por qué seleccionar la actividad que termina antes deja más posibilidades?
- Justifica la optimalidad mediante un argumento de intercambio.
- ¿Qué podría suceder si se eligiese primero la actividad que comienza antes?
- ¿Qué podría suceder si se eligiese primero la actividad de menor duración?
- ¿Cuál es la complejidad temporal y espacial? ¿Qué operación determina la complejidad total?

III · Planificación de tareas con fechas límite

Maximizar el beneficio con tareas unitarias y plazos de finalización.

```
Tarea = (identifier, profit, deadline)
```

Supuestos

- Cada tarea dura un intervalo.
- Se ejecuta como máximo una vez.
- Solo da beneficio si acaba dentro de su deadline.

Regla voraz

1. Ordena por beneficio decreciente.
2. Busca el último intervalo libre ≤ deadline.
3. Descarta la tarea si no queda ninguno.

Empates: deadline menor y luego identificador lexicográfico.

La tarea ocupa un único intervalo y los intervalos se numeran desde 1.

III.A.1 · job_scheduling

El orden de las posiciones de schedule representa los intervalos 1, 2, ..., max deadline.

```
job_scheduling(jobs: List[Tuple[str, int, int]]) -> Tuple[List[Optional[Tuple[str, int, int]]], int]
```

Salida exigida

(schedule, total_profit)

- Huecos: None.
- Longitud: máximo deadline.
- Lista vacía: ([], 0).

Desempates y descarte

Ordenar por profit descendente. Si empatan: deadline ascendente y identifier en orden lexicográfico.

Si no existe un intervalo libre válido, descartar esa tarea.

```
schedule[0] representa el intervalo 1
```

Ejemplo paso a paso · planificador de tareas

Entrada: A(100,2), B(19,1), C(27,2), D(25,1), E(15,3).

```
Orden por beneficio: A(100,2), C(27,2), D(25,1), B(19,1), E(15,3)
```



último libre ≤ 2 | último libre ≤ 2 | último libre ≤ 3

D y B se descartan: no queda intervalo libre dentro de su deadline.

```
schedule = [C, A, E]     ·     total_profit = 27 + 100 + 15 = 142
```

III · También puede formularse con programación dinámica

La práctica pide implementar el algoritmo codicioso. Esta DP es una alternativa exacta para analizar el mismo problema.

Estado

D[i,t] = máximo beneficio usando t tareas entre las primeras i, ordenadas por deadline, con planificación factible.

Decisión

Omitir la tarea i: D[i−1,t].
Incluirla al final: D[i−1,t−1] + $profit_i$, si t ≤ $deadline_i$.

$$D[i,t] = \max(D[i-1,t],\ D[i-1,t-1] + profit_i) \quad \text{si } t \le deadline_i$$

Casos base

D[0,0] = 0; D[0,t>0] = −∞. La respuesta es $\max_t D[n,t]$.

Complejidad

Con D = deadline máximo: O(nD) tiempo y O(nD) espacio para guardar la matriz y reconstruir.

La DP complementa la explicación; la entrega de job_scheduling sigue el criterio voraz indicado.

DP tareas · ejemplo 1 del enunciado

Procesamos las tareas por deadline ascendente: B, D, A, C, E. Columnas = número de tareas elegidas.

`D[i,t] = mejor beneficio al considerar i tareas y elegir t; "−∞" indica que esa cantidad de tareas no cabe.`

Tarea añadida	0	1	2	3
Ninguna	**0**	−∞	−∞	−∞
B: 19, d=1	0	19	−∞	−∞
D: 25, d=1	0	25	−∞	−∞
A: 100, d=2	0	**100**	125	−∞
C: 27, d=2	0	100	**127**	−∞
E: 15, d=3	0	100	127	**142**

Reconstrucción

La ruta marcada selecciona A, C y E: 100 + 27 + 15 = 142.

Asignación válida

A, C, E pueden ocupar los intervalos 1, 2 y 3.

DP tareas · ejemplo 2 con dos plazos

Tareas: X(100,1), Y(70,1), Z(80,2). La capacidad del plazo 1 es un solo intervalo.

`D[i,t] = max(omitir i, incluir i si t ≤ deadline_i)` · filas por deadline y luego identificador.

Tarea añadida	0	1	2
Ninguna	**0**	−∞	−∞
X: 100, d=1	0	**100**	−∞
Y: 70, d=1	0	100	−∞
Z: 80, d=2	0	100	**180**

Reconstrucción

D[3,2] = 180 = 100 + 80: seleccionar X y Z. Se asignan a los intervalos 1 y 2.

Lectura del plazo

Y no mejora D[2,1]: X aporta 100 frente a 70. La DP conserva la mejor opción para cada estado.

Resultado óptimo: X en el intervalo 1 y Z en el intervalo 2; beneficio total 180.

III.B.1 · generate_jobs

Generar tareas reproducibles y con deadlines coherentes con cada tamaño.

```
generate_jobs(n: int, seed: int = 42) -> List[Tuple[str, int, int]]
```

Valores

- profit pertenece a [1, 1000].
- deadline pertenece a [1, max(1, n // 10)].
- Identificadores: J0, J1, ..., J(n−1).

Semilla reproducible

seed permite volver a generar la misma muestra.

Comprueba que cada tupla respeta el orden (identifier, profit, deadline).

III.C · análisis experimental

Medir tiempo, beneficio total y proporción de tareas seleccionadas.

Tamaños a ejecutar

100 · 500 · 1 000 · 5 000 tareas

Plazo máximo derivado de n: max(1, n // 10).

Rangos de deadline del enunciado

100 → 1–10
500 → 1–50
1 000 → 1–100
5 000 → 1–500

Gráfica 1

Número de tareas frente al tiempo.

Gráfica 2

Número de tareas frente al beneficio total.

Gráfica 3

Número de tareas frente al porcentaje seleccionado.

Preguntas para la memoria · tareas

Relaciona los datos generados con el beneficio y la ocupación de intervalos.

- ¿Cuál es el criterio codicioso utilizado?
- ¿Por qué se procesan primero las tareas con mayor beneficio?
- ¿Por qué se asigna cada tarea al último intervalo disponible?
- ¿Cómo afectan la distribución de deadlines y la distribución de beneficios?
- ¿Qué podría suceder si se ordenasen únicamente por deadline?
- ¿Cuál es la complejidad temporal y espacial? ¿Cómo podría mejorarse la búsqueda del último intervalo libre?

IV · cuestiones generales

Compara los tres algoritmos y explica qué propiedades respaldan sus decisiones.

- ¿Qué rasgos permiten afirmar que los tres algoritmos estudiados son codiciosos?
- ¿En qué problemas se garantiza una solución óptima?
- ¿En qué problema puede obtenerse una solución subóptima?
- ¿Por qué no basta con que una decisión parezca localmente óptima?
- ¿Qué diferencias hay entre los tres algoritmos en complejidad temporal y espacial?
- ¿Qué ventajas y limitaciones presenta el paradigma codicioso frente a la programación dinámica?

Una guía para comparar complejidades

La respuesta debe indicar qué operación domina y bajo qué representación.

Problema	Operación clave	Tiempo orientativo	Espacio orientativo
Cambio codicioso	Ordenar/recorrer monedas y emitir k monedas	O(m log m + k) al ordenar	O(k) salida
Cambio óptimo	Probar m monedas para cada cantidad	O(A·m)	O(A) DP + salida
Actividades	Ordenar n intervalos y recorrerlos	O(n log n)	O(n) con copia y salida
Tareas	Ordenar n y buscar slot entre D posiciones	O(n log n + nD)	O(n + D)

m: número de denominaciones · k: monedas devueltas · A: cantidad objetivo · n: actividades/tareas · D: máximo deadline. El coste de salida también cuenta.

V · Qué hay que entregar

Dos ficheros, con contenido y finalidad distintos.

p2.py

Funciones del apartado I:
- greedy_change
- optimal_change
- compare_change

Del apartado II:
- activity_selection
- generate_activities

Del apartado III:
- job_scheduling
- generate_jobs

memoria.pdf

Respuestas razonadas a las cuestiones de I, II, III y IV.

Incluye los resultados por sistema monetario, la tabla resumen y la comparación de tiempos.

Añade las gráficas solicitadas para actividades y tareas, con ejes, unidades y análisis.

p2.py no debe incluir scripts de prueba

Checklist antes de entregar

Verifica implementación, experimentos y memoria.

- Las siete funciones tienen exactamente las firmas requeridas.
- greedy_change informa el resto; optimal_change devuelve lista o None.
- compare_change clasifica todas las cantidades de 1 a max_amount.
- activity_selection no modifica la entrada y aplica el desempate indicado.
- job_scheduling usa None para huecos y la posición 0 para el intervalo 1.
- Las semillas permiten reproducir las entradas experimentales.
- La memoria responde todas las preguntas y justifica costes y resultados.
- Las gráficas identifican ejes, unidades, tamaños y medidas.

Objetivo: código correcto, análisis razonado y entrega completa